SYSTEM DESIGN REVISION GUIDE

A Trade-Off Driven Reference on Distributed Systems, Scalability Patterns, API Gateways & Fault Tolerance

A Concise, Visual, and Mechanism-Focused Interview & Revision Handbook

METADATA FIELD	SPECIFICATION
Subject:	System Design & Distributed Architectures
Target Audience:	SDE / Technical Interviews (Top Tech, FAANG, Product Firms)
Methodology:	Internal Mechanics • State Transitions • Verified Code Recipes • Trade-offs
Source File Consolidated:	Interview Concepts/System_Design.py (HLD vs LLD foundations, Parking Lot example, API Gateway)
Subject Coverage:	HLD vs LLD, Scaling, Load Balancers, API Gateways, Distributed Caching, Message Queues, Rate Limiting, CAP
Key Architectural Focus:	Trade-off Matrices, Multi-Tier Architecture Blueprints, Rate Limiting Algorithms & Failure Recovery
Target Evaluation:	Technical Interview Placement Preparation - Core Engineering Track

CH	CHAPTER TITLE & CORE MECHANISM	CRITICAL INTERVIEW CONCEPTS & TRADE-OFFS
1	System Design Foundations & Trade-offs	Functional vs Non-Functional Requirements, Latency vs Throughput, Availability SLOs
2	High-Level Design (HLD) vs Low-Level Design (LLD)	Architecture vs Object modeling, Parking Lot case study, Component decoupling
3	Scaling Foundations: Vertical vs Horizontal	Scale-Up bottlenecks vs Scale-Out stateless servers, Shared-Nothing architecture
4	Load Balancing & Reverse Proxies	Layer 4 vs Layer 7, Nginx/HAProxy, Round Robin, Least Connections, IP Hash
5	API Gateway Architecture	Single entry point, Authentication, Rate Limiting, SSL Termination, Request Routing
6	Multi-Tier Distributed Caching	CDN, API Gateway, Redis in-memory cache, DB buffer pool, Cache invalidation
7	Distributed Data Storage & Sharding	Leader-Follower, Read Replicas, Horizontal Sharding, Consistent Hashing ring
8	Asynchronous Processing & Message Queues	Kafka vs RabbitMQ, Decoupled Producers/Consumers, Backpressure, Delivery semantics
9	Rate Limiting Algorithms	Token Bucket, Leaky Bucket, Fixed Window, Sliding Window Counter
10	High Availability, CAP & Circuit Breakers	CAP Theorem, PACELC Theorem, Circuit Breakers (Closed/Open/Half-Open states)

CHAPTER 1 - SYSTEM DESIGN FOUNDATIONS & TRADE-OFFS

System Design is the process of defining architecture, components, modules, interfaces, and data for a system to satisfy specified requirements. There is rarely a single 'correct' architecture; every decision represents a calculated trade-off between competing constraints.

1.1 Functional vs Non-Functional Requirements

Category	Definition & Focus	Typical Examples in Interviews
Functional Requirements (FR)	What the system **MUST DO** (features, business logic, user operations).	'Users can upload videos', 'Users can search tweets', 'System generates short URLs'.
Non-Functional Requirements (NFR)	How well the system **PERFORMS** under operational load.	High Availability (99.99%), Low Latency (<100ms p99), High Throughput, Fault Tolerance.

1.2 The Numbers Everyone Should Know (Latency Hierarchy)

Hardware Operation	Approximate Real-Time Latency	Human-Scale Analogy (1ns = 1 sec)
L1 CPU Cache reference	0.5 ns	0.5 seconds
L2 CPU Cache reference	7 ns	7 seconds
Main Memory (RAM) reference	100 ns	1.6 minutes
NVMe SSD Random Read	150,000 ns (150 us)	1.7 days
Rotational HDD Disk Seek	10,000,000 ns (10 ms)	3.8 months
Packet Round Trip (US to Europe)	150,000,000 ns (150 ms)	4.7 years!

CHAPTER 2 - HIGH-LEVEL DESIGN (HLD) VS LOW-LEVEL DESIGN (LLD)

Modern technical interviews split system design into High-Level Design (Macro Architecture) and Low-Level Design (Object/Class Modeling).

Dimension	High-Level Design (HLD)	Low-Level Design (LLD)
Core Question	*'What major components exist, and how do they communicate?'*	*'How does a specific component work internally?'*
Scope of Focus	Servers, Load Balancers, API Gateways, Caches, DBs, Message Queues.	Classes, Interfaces, Methods, Inheritance, Design Patterns.
Primary Units	Microservices, Data pipelines, Distributed storage nodes.	Objects, Data models, In-memory structures, Algorithms.
Design Principles	Scalability, Fault tolerance, High Availability, CAP theorem.	SOLID principles, OOP patterns (Factory, Strategy, Observer).
Case Study: Parking Lot	User -> Load Balancer -> API Gateway -> Parking Service -> Redis -> PostgreSQL -> Kafka.	Classes: ParkingLot, Floor, ParkingSpot (Compact, Large), Vehicle, Ticket, Payment.

CHAPTER 3 - SCALING FOUNDATIONS: VERTICAL VS HORIZONTAL

Dimension	Vertical Scaling (Scale-Up)	Horizontal Scaling (Scale-Out)
Method	Adding more CPU cores, RAM, and faster NVMe storage to a single server.	Adding more discrete commodity server nodes to a cluster.
Hardware Limit	Strict physical hardware ceiling (maximum motherboard sockets/RAM).	Practically infinite; scale by adding additional nodes.
Downtime Risk	Single Point of Failure (SPOF); scaling often requires maintenance reboots.	High Availability: if one node dies, others absorb traffic seamlessly.
Complexity	Simple: code runs unchanged; zero distributed concurrency complexity.	Complex: requires Load Balancers, stateless application design, service discovery.
Cost Scaling	Exponential: high-end enterprise server parts cost drastically more per flop.	Linear: uses affordable standard commodity cloud instances.

CHAPTER 4 - LOAD BALANCING & REVERSE PROXIES

4.1 Layer 4 vs Layer 7 Load Balancing

Feature	Layer 4 Load Balancer (Transport)	Layer 7 Load Balancer (Application)
Inspection Level	Inspects IP address and TCP/UDP port only.	Inspects full HTTP payload: Headers, Cookies, URL path, Body.
Performance & CPU	Blazing fast; minimal CPU overhead (routes raw packets).	Slightly higher latency; must decrypt TLS and parse HTTP.
Smart Routing	Cannot route based on content (e.g., path `/video` vs `/chat`).	**Content-based routing:** sends `/api/payment` to Payment Service and `/static` to CDN.

4.2 Load Balancing Algorithms

Algorithm	Selection Rule	Best Suited For
Round Robin	Sequentially cycles through all available server nodes.	Identical servers with uniform, short request durations.
Weighted Round Robin	Cycles according to assigned server capacity weights.	Heterogeneous server hardware with unequal RAM/CPU.
Least Connections	Routes traffic to the node currently handling the fewest active connections.	Long-running requests (e.g., file downloads, WebSocket sessions).
IP Hash	Hashes client IP to assign a persistent deterministic server.	Stateful applications requiring rudimentary session stickiness.

CHAPTER 5 - API GATEWAY ARCHITECTURE

An **API Gateway** acts as the single reverse proxy entry point for all client requests in a microservices architecture. It decouples client applications from internal microservice boundaries.

Core Duty	Technical Implementation Mechanism	Architectural Value
Request Routing	Inspects URL path (e.g., `/auth/*`, `/orders/*`) and forwards to downstream service.	Hides internal microservice topology and internal ports from public Internet.
Authentication & Auth	Validates JWT tokens, API keys, and session cookies at the edge.	Downstream microservices receive trusted pre-authenticated user IDs.
Rate Limiting	Enforces request quotas per IP / API key via Redis token buckets.	Protects backend services against DDoS attacks and brute-force scraping.
SSL Termination	Decrypts inbound HTTPS traffic and speaks plain HTTP internally.	Offloads heavy cryptographic CPU processing from application servers.
Observability & Logging	Injects unique correlation IDs (`X-Request-ID`) into headers.	Enables distributed tracing across dozens of microservices.

CHAPTER 6 - MULTI-TIER DISTRIBUTED CACHING

Tier Level	Caching Location	Stored Data	Latency
Tier 1: Client	Browser memory / LocalStorage	Static assets, session tokens, user profile JSON.	0 ms (Local RAM)
Tier 2: CDN Edge	Cloudflare, Akamai, CloudFront edge PoPs	Images, CSS/JS bundles, video chunks, public API responses.	10 - 30 ms
Tier 3: API Gateway	Gateway reverse proxy memory (Nginx/Kong)	Frequent, read-heavy aggregate responses.	1 - 5 ms
Tier 4: Distributed Cache	Redis / Memcached dedicated cluster	Database query results, user sessions, shopping carts.	0.5 - 2 ms
Tier 5: Database Cache	PostgreSQL shared_buffers / InnoDB buffer pool	Frequently accessed disk database pages in DB server RAM.	0.1 - 0.5 ms

CACHE INVALUATION TRADEOFF

'There are only two hard things in Computer Science: cache invalidation and naming things.' - Phil Karlton

Always define explicit TTLs on all cache tiers. Combine TTLs with event-driven invalidation (e.g., publishing DB mutation events to Redis Pub/Sub to evict stale CDN/Redis keys).

CHAPTER 7 - DISTRIBUTED DATA STORAGE & SHARDING

7.1 Horizontal Sharding & Consistent Hashing

The Problem with Simple Modulo: If you shard using server_id = hash(key) % N, adding or removing a single server changes N. This forces almost 100% of all keys to relocate, triggering a massive database cache avalanche and network storm!

The Solution: Consistent Hashing:

• Both keys and server nodes are hashed onto a conceptual 360-degree circular ring (0..2^32 - 1).

• A key is assigned to the first server encountered moving clockwise on the ring.

• **Virtual Nodes:** Each physical server is assigned 100-200 virtual node positions on the ring. This guarantees uniform key distribution and eliminates hotspots. When a node is added/removed, only K / N keys move!

7.2 Master-Slave vs Multi-Master Replication

Replication Topology	Write Path	Read Path	Conflict Risk
Master-Slave (Single Leader)	Writes routed exclusively to Leader node.	Reads load-balanced across multiple Follower replicas.	Zero write conflicts; potential read replication lag.
Multi-Master (Multi-Leader)	Writes accepted at multiple geographically separated leaders.	Reads served locally at closest leader/replica.	High: Requires conflict resolution (Last-Write-Wins or CRDTs).
Leaderless (Dynamo / Cassandra)	Client writes to multiple nodes concurrently using Quorum: W + R > N.	Client reads from multiple nodes and repairs inconsistent versions.	Tunable consistency; no single point of failure.

CHAPTER 8 - ASYNCHRONOUS PROCESSING & MESSAGE QUEUES

8.1 Synchronous vs Asynchronous Communication

Synchronous HTTP/REST calls couple services tightly: if Service B is slow or down, Service A hangs and exhausts its connection pool. A **Message Queue** buffers events asynchronously, providing temporal decoupling and load leveling (backpressure absorption).

Feature	RabbitMQ (AMQP Message Broker)	Apache Kafka (Distributed Event Log)
Architecture	Traditional message queue: pushes messages to consumers.	Distributed append-only commit log: consumers pull at their own pace.
Message Retention	Messages deleted immediately after consumer acknowledgment.	Messages persisted on disk for configured retention period (e.g., 7 days).
Ordering Guarantee	Order preserved per queue, but disrupted by consumer acknowledgments.	Strict FIFO order guaranteed **per partition**.
Throughput Profile	Moderate throughput (~50K msg/sec); complex routing rules (exchanges).	Ultra-high throughput (1M+ msg/sec); zero-copy sequential disk I/O.
Ideal Use Case	Task queues (e.g., Celery, PDF generation, transactional alerts).	Event sourcing, log aggregation, real-time analytics streaming.

CHAPTER 9 - RATE LIMITING ALGORITHMS

Algorithm	Internal Mechanism	Memory Cost	Burst Handling & Accuracy
Token Bucket	Tokens added to bucket at constant rate r up to capacity b. Request consumes 1 token.	O(1) (tracks tokens and timestamp).	Allows bursts up to capacity b; smooth average rate. Standard in AWS/Stripe.
Leaky Bucket	Requests enter a FIFO queue and leak out at a constant rate.	O(Capacity) (stores request queue).	Zero burst tolerance; strictly smooths traffic into a uniform output stream.
Fixed Window Counter	Counts requests in fixed time slices (e.g., 100 req / minute).	O(1) per user per window.	**Boundary Spike Vulnerability:** 200 requests can squeeze in at window boundary!
Sliding Window Counter	Combines current window count with weighted percentage of previous window.	O(1) per user.	Smooths boundary spikes with minimal memory overhead.

CHAPTER 10 - HIGH AVAILABILITY, CAP & CIRCUIT BREAKERS

10.1 Circuit Breaker Pattern (Netflix Hystrix)

In a distributed system, a failing downstream microservice causes upstream services to block waiting for timeouts, exhausting server threads and triggering a **cascading system-wide failure**.

• **CLOSED:** Normal operation. Requests flow to downstream service. If error rate exceeds threshold (e.g., 50%), circuit trips to OPEN.

• **OPEN:** Fail Fast! Requests immediately return a fallback error/cached response without calling downstream service. Prevents resource exhaustion.

• **HALF-OPEN:** After a sleep window, a trial batch of requests is allowed through. If successful, circuit resets to CLOSED. If it fails, resets to OPEN.

10.2 PACELC Theorem (Extended CAP)

CAP only describes behavior during rare network partitions. PACELC extends CAP to normal operation:

*'If there is a **P**artition, trade **A**vailability vs **C**onsistency; **E**lse (normal operation), trade **L**atency vs **C**onsistency.'*

• **PC/EC (e.g., BigTable, Spanner):** Consistent under partition; low latency sacrificed for consistency in normal mode.

• **PA/EL (e.g., Cassandra, DynamoDB):** Available under partition; yields lower latency over strict consistency in normal mode.

SYSTEM DESIGN INTERVIEW BLUEPRINT

1. **Scope Requirements:** Clarify Functional (core APIs) and Non-Functional (scale, QPS, latency, availability).
2. **Back-of-Envelope Math:** Daily Active Users (DAU) -> Read QPS & Write QPS -> Storage per year -> Network Bandwidth.
3. **High-Level Blueprint:** Draw Client -> CDN/DNS -> Load Balancer -> API Gateway -> Stateless App Servers -> Cache -> DB.
4. **Deep Dive & Bottlenecks:** Sharding keys, replication lag, rate limiting, and failure recovery.